1、v-if 和 v-for 哪个优先级高

1. vue2 中，v-for 优先于 v-if，vue3 中 v-if 的优先级高于 v-for
2. 如果同时出现，每次渲染都会先执行循环在判断条件，无论如何，循环都不可避免，浪费了性能
3. 要避免出现这种情况，可以在外层嵌套 template，在这一层加 v-if，然后内部进行 v-for 循环
4. 如果条件出现在循环内部，可通过计算属性提前过滤掉那些不需要显示的项

源码中找答案 compiler/codegen/index.js

```
// Vue2 源码相关处
if (el.staticRoot && !el.staticProcessed) {
    return genStatic(el, state)
} else if (el.once && !el.onceProcessed) {
    return genOnce(el, state)
} else if (el.for && !el.forProcessed) {
    return genFor(el, state)
} else if (el.if && !el.ifProcessed) {
    return genIf(el, state)
} else if (el.tag === 'template' && !el.slotTarget && !state.pre) {
    return genChildren(el, state) || 'void 0'
} else if (el.tag === 'slot') {
    return genSlot(el, state)
} else {}

// Vue3 源码相关处
switch (node.type) {
  case NodeTypes.ELEMENT:
  case NodeTypes.IF:
  case NodeTypes.FOR:
    __DEV__ &&
      assert(
        node.codegenNode != null,
        `Codegen node is missing for element/if/for node. ` +
          `Apply appropriate transforms first.`
      )
    genNode(node.codegenNode!, context)
    break
  case NodeTypes.TEXT:
    genText(node, context)
    break
  ...
  ...
}
```

2、组件 data 为什么必须是个函数而根实例则没有此限制

1. vue 组件可能存在多个实例，如果使用对象形式定义 data，则会导致它们共用一个 data 对象，那么状态变更将会影响所有组件实例，这是不合理的
2. 采用函数形式定义，在 initData() 时会将其作为工厂函数返回全新 data 实例，有效规避多实例之间状态污染问题
3. vue 根实例创建过程中，则不存在该限制，也是因为根实例只有一个，不需要担心这种情况。

源码中找答案：src\core\instance\state.js – initData()

3、key 的作用和工作原理吗?

源码中找答案：src\core\vdom\patch.js – updateChildren()

1. key 的作用主要就是为了高效的更新DOM，其原理是vue在patch过程中通过key可以精准判断两个节点是否是同一个，从而避免频繁的更新相同元素，使得整个patch过程更加高效，减少DOM操作量，提高性能。
2. 如果不设置key，还可能在列表更新时引发一些隐藏bug
3. vue中在使用相同标签名元素的过度切换时，也会使用到key属性，其目的也是为了让vue可以区分它们，否则vue 只会替换其内部属性而不会触发过度效果。

4、diff 算法?

1. diff 算法是虚拟 DOM 技术的必然产物：
    - 通过新旧虚拟 DOM 做对比，将变化的地方更新在真实 DOM 上，另外，也需要 diff 高效的执行对比过程，从而降低时间复杂度为 O(n)
2. vue2 中为了降低 Watcher 粒度，每个组件只有一个 Watcher 与之对应，只有引入 diff 才能精确找到发生变化的地方。
3. vue 中 diff 执行的时刻是组件实例执行更新函数时，它会比对上一次的渲染结果 oldVnode 和新渲染结果 newVnode，此过程称之为 patch
4. diff 过程整体遵循深度优先、同层比较的策略。
    - 两个节点之间比较会根据它们是否拥有子节点或者文本节点做不同操作。
    - 比较两组子节点是算法的重点，首先假设头尾节点可能相同做 4 次比对尝试，
    - 如果没有找到相同节点才按照通用方式遍历查找，查找结束再按情况处理剩下的节点，
    - 借助 key 通常可以非常精确的找到相同节点，因此整个 patch 过程非常高效。

- 源码分析1：必要性，lifecycle.js – mountComponent()
    - 组件中可能存在很多个 data 中的 key 使用
- 源码分析2：执行方式，patch.js – patchVnode()
    - patchVnode 是 diff 发生的地方，整体策略：深度优先，同层比较
- 源码分析3：高效性，patch.js – updateChildren()

5、组件化的理解

源码分析1：组件定义，src\core\global-api\assets.js

vue-loader会编译template为render函数，最终导出的依然是组件配置对象。

源码分析2：组件化优点，lifecycle.js - mountComponent()

组件、Watcher、渲染函数和更新函数之间的关系

源码分析3：组件化实现，构造函数，src\core\global-api\extend.js 实例化及挂载，src\core\vdom\patch.js - createElm()

1. 组件是独立和可复用的代码组织单元，组件系统是Vue核心特性之一，它使开发者使用小型、独立和通常可复用的组件构建大型应用。
2. 组件化开发能大幅度提高应用开发效率，测试性，复用性等。
3. 组件使用按分类有：页面组件、业务组件、通用组件。
4. vue的组件是基于配置的，我们通常编写的组件是组件配置而非组件，框架后续会生成其构造函数，他们基于VueComponent，扩展于vue
5. vue中常见的组件化技术有：属性prop，自定义事件，插槽等，它们主要用于组件通信、扩展等
6. 合理的划分组件，有助于提升应用性能
7. 组件应该是高内聚，低耦合的。
8. 遵循单向数据流的原则

6、Vue 设计原则的理解

1. 渐进式 JS 框架

- vue 被设计为可以自底向上逐层应用。
- vue 的核心库只关注视图层，不仅易于上手，还便于与第三方库或既有项目整合。
- 当与现代化工具链以及各种支持类库结合使用时，Vue 也完全能够为复杂的单页应用提供驱动。

2. 易用性

- vue 提供数据响应式、声明式模版语法和基于配置的组件系统等核心特性
- 使我们只关注应用的核心业务即可，只会写 js，css，html 就能轻松过编写 vue 应用

3. 灵活性

- 渐进式框架最大的优点就是灵活性，如果应用足够小，我们可能仅需要 vue 核心特性即可完成功能
- 随着应用不断扩大，才可能逐渐引入路由，状态管理，等工具和库，不管是应用体积还是学习难度都是一个逐渐增加的平和曲线

4. 高效性

- 超快的虚拟 DOM 和 diff 算法使我们的应用拥有极佳的性能表现

- 追求高效的过程还在继续，`vue3` 中引入 `proxy` 对数据响应式改进以及编译器中对于静态内容编译的改进都会让 vue 更加高效

7、MVC、MVP和MVVM的理解

1. 这三个都是框架模式，它们设计的目标都是解决Model和View的耦合问题
2. MVC模式出现比较早，主要应用在后端，在前端领域的早期也有应用，如Backbone.js。它的优点是分层清晰，缺点是数据流混乱，灵活性带来的维护性问题
3. MVP模式是MVC的进化形式，Presenter作为中间层负责MV通信，解决了两者耦合问题，但P层过于臃肿会导致维护问题
4. MVVM模式在前端领域有广泛的应用，它不仅解决MV耦合问题，还同时解决了维护两者映射关系的大量繁杂代码和DOM操作代码，在提高开发效率、可读性同时还保持了优越的性能表现。

8、vue 性能优化的方法

1. 路由懒加载
2. keep-alive 缓存页面
3. v-show 复用 DOM
4. v-for 遍历避免同时使用 v-if
5. 长列表性能优化
    - 纯数据，非响应式的可以 使用 object.freeze 冻结
    - 大数据长列表，可采用虚拟滚动，只渲染少部分内容 （vue-virtual-scroller、vue-virtual-scroll-list）
6. 事件的销毁
7. 图片的懒加载 （vue-lazyload）
8. 第三方插件按需引入
9. 无状态组件标记为函数组件 functional
10. 使用服务器端渲染：使用服务器端渲染（SSR）来生成 HTML，以减少客户端渲染所需的时间和资源。但需要注意，SSR 也可能增加了服务器的负担并使网站更复杂。(SEO、首屏渲染)
11. 变量本地化
12. 使用代码分割、子组件分割：将代码拆分成小块并按需加载（懒加载），以避免不必要的网络请求和减少加载时间。
13. 缓存资源：利用浏览器缓存来存储重复使用的文件，例如 CSS 和 JS 文件、图片等。
14. 预加载关键资源：在首次渲染之前，先提前加载关键资源，例如首页所需的 JS、CSS 或数据，以保证关键内容的快速呈现。
15. 使用合适的图片格式：选择合适的图片格式（例如 JPEG、PNG、WebP 等），并根据需要进行压缩以减少文件大小。对于一些小图标，可以使用 `iconfont` 等字体文件来代替。
16. 启用 Gzip 压缩：使用服务器端的 Gzip 压缩算法对文件进行压缩，以减少传输时间和带宽消耗。
17. 使用 CDN：使用内容分发网络（CDN）来缓存和传递文件，以提高文件的下载速度和可靠性。
18. 优化 API 请求：尽可能地减少 API 调用的数量，并使用缓存和延迟加载等技术来优化 API 请求的效率。

9、model 和 event.target.value

当 input 事件是由 IME （即由输入法触发）构成触发的，会直接 return，不再获取值。

`v-model` 是 `value` 和 `oninput` 事件的结合，能够动态地对 value 进行改变，若是 value 被改变了，能够很快地反映到对应的组件当中，改变该组件的 value

10、双向绑定使用和原理

1. vue 中双向绑定是一个指令 `v-model` ，可以绑定一个响应式数据到视图，同时视图中变化能改变该值。
2. **v-model 是语法糖，默认情况下相当于 :value 和 @input 。** 使用 v-model 可以减少大量繁琐的事件处理代码，提高开发效率。
3. 通常在表单项上使用 v-model ，还可以在自定义组件上使用，表示某个值的输入和输出控制。
4. 通过 `<input v-model="xxx">` 的方式将 xxx 的值绑定到表单元素 value 上；
    - 对于 checkbox，可以使用 true-value 和 false-value 指定特殊的值，
    - 对于 radio 可以使用 value 指定特殊的值；
    - 对于 select 可以通过 options 元素的 value 设置特殊的值；
    - 还可以结合 .lazy, .number, .trim 对 v-model 的行为做进一步限定；
    - v-model 用在自定义组件上时又会有很大不同，
    - vue3 中它类似于 sync 修饰符，最终展开的结果是 modelValue 属性和 update:modelValue 事件；
    - vue3 中甚至可以用参数形式指定多个不同的绑定，例如 v-model:foo 和 v-model:bar，非常强大！
5. v-model 是一个指令，它的神奇魔法实际上是 vue 的编译器完成的。
    - 我做过测试，包含 v-model 的模板，转换为渲染函数之后，实际上还是 value 属性的绑定以及 input 事件监听，事件回调函数中会做相应变量更新操作。
    - 编译器根据表单元素的不同会展开不同的 DOM 属性和事件对，比如 text 类型的 input 和 textarea 会展开为 value 和 input 事件；
    - checkbox 和 radio 类型的input 会展开为 checked 和 change 事件；
    - select 用 value 作为属性，用change 作为事件。
6. `console.log(app.\$options.render)`; 就可以输出看到结果。

11、vue 响应式原理的理解?

1、object.defineProperty

2、proxy(兼容性不太好)

Object.defineProperty 有哪些缺陷，Vue3 为什么要用 proxy 重构

- 不能监听数组的变化 重写数组方法
- 不能监听对象属性新增和删除
- 深层监听困难

- 初始化阶段递归执行 Object.defineProperty 带来的性能负担

observer类

```
/* observer 类会附加到每一个被侦测的object上
* 一旦被附加上，observer会被object的所有属性转换为getter/setter的形式
* 当属性发生变化时候及时通知依赖
*/
// Observer 实例
export class Observer {
  constructor(value) {
    this.value = value
    if (!Array.isArray(value)) { // 判断是否是数组
      this.walk(value) // 劫持对象
    }
  }
  walk(obj) {
    // 将会每一个属性转换为getter/setter 形式来侦测数据变化
    const keys = Object.keys(obj)
    for (let i = 0; i < keys.length; i++) {
      defineReactive(obj, keys[i], obj[key[i]]) // 数据劫持方法
    }
  }
}

function defineReactive(data, key, val) {
  // 递归属性
  if (typeof val === 'object') {
    new Observer(val)
  }
  let dep = new Dep()
  Object.defineProperty(data, key, {
    enumerable: true,
    configurable: true,
    get: function () {
      dep.depend()
      return val
    },
    set: function (newVal) {
      if (val === newVal) {
        return
      }
      val = newVal
      dep.notify()
    }
  })
}
```

Dep 依赖收集

```
export default class Dep {
  constructor() {
    this.subs = [] // 观察者集合
  }
  // 添加观察者
  addSub(sub) {
    this.subs.push(sub)
  }
  // 移除观察者
  removeSub(sub) {
    remove(this.subs, sub)
  }
  depend() {
    // 核心，如果存在 ，则进行依赖收集操作
    if (window.target) {
      this.addDep(window.target)
    }
  }
  notify() {
    const subs = this.subs.slice() // 避免污染原来的集合
    // 如果不是异步执行，先进行排序，保证观察者执行顺序
    if (process.env.NODE_ENV !== 'production' && !config.async) {
      subs.sort((a, b) => a.id - b.id)
    }
    for (let i = 0, l = subs.length; i < l; i++) {
      subs[i].update() // 发布执行
    }
  }
}
function remove(arr, item) {
  if (arr.length) {
    const index = arr.indexOf(item)
    if (index > -1) {
      return arr.splice(index, 1)
    }
  }
}
```

watcher

```
export default class Watcher {
  constructor(vm, expOrFn, cb) {
    // 组件实例对象
    // 要观察的表达式，函数，或者字符串，只要能触发取值操作
    // 被观察者发生变化后的回调
    this.vm = vm // Watcher有一个 vm 属性，表明它是属于哪个组件的
    // 执行this.getter()及时读取数据
    this.getter = parsePath(expOrFn)
    this.cb = cb
    this.value = this.get()
```

```
  }
  get() {
    window.target = this
    let value = this.getter.call(this.vm, this.vm)
    window.target = undefined
    return value
  }
  update() {
    const oldValue = this.value
    this.value = this.get()
    this.cb.call(this.vm, this.value, oldValue)
  }
}
```

```
* data通过Observer转换成了getter/setter的形式来追踪变化
* 当外界通过Watcher读取数据时，会触发getter从而将watcher添加到依赖中
* 当数据变化时，会触发setter从而向Dep中的依赖（watcher）发送通知
* watcher接收通知后，会向外界发送通知，变化通知到外界后可能会触发视图更新，也有可能触发用户的某个回调函数等
```

1. 啥是响应式?

所谓数据响应式就是能够使数据变化可以被检测并对这种变化做出响应的机制。

2. 为什么vue需要响应式?

MVVM 框架中要解决的一个核心问题是连接数据层和视图层，通过数据驱动应用，数据变化，视图更新，要做到这点的就需要对数据做响应式处理，这样一旦数据发生变化就可以立即做出更新处理。

3. 它能给我们带来什么好处?

以vue为例说明，通过数据响应式加上虚拟DOM和patch算法，开发人员只需要操作数据，关心业务，完全不用接触繁琐的DOM操作，从而大大提升开发效率，降低开发难度。

4. vue的响应式是怎么实现的？有哪些优缺点?

vue2 中的数据响应式会根据数据类型来做不同处理，

- 如果是对象则采用 `Object.defineProperty()` 的方式定义数据拦截，当数据被访问或发生变化时，我们感知并作出响应；
- 如果是数组则通过覆盖数组对象原型的 7 个变更方法，使这些方法可以额外的做更新通知，从而作出响应。
- 这种机制很好的解决了数据响应化的问题，但在实际使用中也存在一些缺点：比如初始化时的递归遍历会造成性能损失；
- 新增或删除属性时需要用户使用 `Vue.set/delete` 这样特殊的 api 才能生效；
- 对于 es6 中新产生的 Map、Set 这些数据结构不支持等问题。

5. vue3 中的响应式的新变化

为了解决这些问题，vue3 重新编写了这一部分的实现：

- 利用 ES6 的 Proxy 代理要响应化的数据，它有很多好处，编程体验是一致的，不需要使用特殊 api，初始化性能和内存消耗都得到了大幅改善；
- 另外由于响应化的实现代码抽取为独立的 reactivity 包，使得我们可以更灵活的使用它，第三方的扩展开发起来更加灵活了。

12、vue 中组件之间的通信方式?

1、父子组件通信

- props
- emit/on
- children/parent
- attrs/listeners
- ref

2、兄弟组件

- $parent
- $root
- eventbus
- vuex

3、跨层级关系

- eventbus
- vuex
- provide/inject

  vue3 中废弃的几个 API

1、在 3.x 中，该`$children`属性已被移除且不再受支持。相反，如果您需要访问子组件实例，建议使用模板 refs 。

2、在 Vue 3 的虚拟 DOM 中，事件监听器现在只是属性，前缀 `on` 为 ，因此是对象的一部分 `$attrs`，因此 `$listeners` 已被删除。

3、从实例中完全删除了 on、off 和 once 方法，emit 仍然是现有 API 的一部分，因为它用于触发父组件声明性附加的事件处理程序。事件总线模式可以通过使用实现事件发射器接口的外部库来替代，例如 mitt 或 tiny emitter。在大多数情况下，不鼓励使用全局事件总线在组件之间进行通信。

13、vue 生命周期 父子组件生命周期

1. 创建加载阶段：父 beforeCreate -> 父 created -> 父 beforeMount -> 子 beforeCreate -> 子 created -> 子 beforeMount -> 子 mounted -> 父 mounted

2. 更新阶段：父 beforeUpdate -> 子 beforeUpdate -> 子 updated -> 父 updated
3. 销毁阶段：父 beforeDestroy -> 子 beforeDestroy -> 子 destroyed -> 父 destroyed

14、nextTick？它是干什么的？实现原理是什么？

1. nextTick 是 Vue 提供的一个全局 API，由于 vue 的异步更新策略导致我们对数据的修改不会立刻体现在 dom 变化上，此时如果想要立即获取更新后的 dom 状态，就需要使用这个方法。
2. Vue 在更新 DOM 时是异步执行的。只要侦听到数据变化，Vue 将开启一个队列，并缓冲在同一事件循环中发生的所有数据变更。如果同一个 watcher 被多次触发，只会被推入到队列中一次。这种在缓冲时去除重复数据对于避免不必要的计算和 DOM 操作是非常重要的。nextTick 方法会在队列中加入一个回调函数，确保该函数在前面的 dom 操作完成后才调用。
3. 所以当我们想在修改数据后立即看到 dom 执行结果就需要用到 nextTick 方法。
4. 比如，我在干什么的时候就会使用 nextTick，传一个回调函数进去，在里面执行 dom 操作即可。
5. nextTick 实现：它会在 callbacks 里面加入我们传入的函数，然后用 timerFunc 异步方式调用它们，首选的异步方式会是 Promise。

拓展：

1. vue 用异步队列的方式来控制 DOM 更新和 nextTick 回调先后执行
2. microtask 因为其高优先级特性，能确保队列中的微任务在一次事件循环前被执行完毕
3. 因为兼容性问题，vue 不得不做了 microtask 向 macrotask 的降级方案

在下次 dom 更新循环结束之后执行延迟回调，可用于获取更新后的 dom 状态

新版本中默认是 microtasks, v-on 中会使用 macrotasks

macrotasks 任务的实现: setImmediate / MessageChannel / setTimeout

```
// <div  ref="div"  style width="width" />
// <button @click="changeWidth" />
// width: 100px

changeWidth(){
    console.log(this.$refs.div.style.width) // 100px
    this.width = 200px
    this.$nextTick(()=>{
      console.log(this.$refs.div.style.width) // 100px
    })
}
mounted() {
  console.log(this.$refs.div.style.width) // 100px
  this.$nextTick(()=>{
    console.log(this.$refs.div.style.width) // 200px
  })
  this.width = '200px'
}
```

```
// 在点击事件里边第一次点击，是不会获取更新数据的，包括nextTick函数中，第二次点击可以得到上次的变化
// mounted 生命周期里边是 可以得到更细数据的
```

15、vue 组件模板只能有一个根元素?

1. new Vue({el:'#app'})
2. 单文件组件中，template 下的元素 div。其实就是"树"状数据结构中的"根"。
3. diff 算法要求的，源码中，patch.js 里 patchVnode()。

16、生命周期

beforeCreate(创建前)

在实例初始化之后，数据观测和事件配置之前被调用，此时组件的选项对象还未创建，el 和 data 并未初始化，因此无法访问 methods， data， computed 等上的方法和数据。

created (创建后)

实例已经创建完成之后被调用，在这一步，实例已完成以下配置：

- 数据观测、属性和方法的运算，
- watch/event事件回调，
- 完成了data 数据的初始化，el 没有。

然而，挂载阶段还没有开始, $el 属性目前不可见，这是一个常用的生命周期，因为你可以调用 methods 中的方法，改变 data 中的数据，并且修改可以通过 vue 的响应式绑定体现在页面上，获取 computed 中的计算属性等等，通常我们可以在这里对实例进行预处理，也有一些童鞋喜欢在这里发 ajax 请求，值得注意的是，这个周期中是没有什么方法来对实例化过程进行拦截的，因此假如有某些数据必须获取才允许进入页面的话，并不适合在这个方法发请求，建议在组件路由钩子 beforeRouteEnter 中完成

beforeMount

挂载开始之前被调用，相关的 render 函数首次被调用（虚拟 DOM），实例已完成以下的配置：

- 编译模板，把 data 里面的数据和模板生成 html，完成了 el 和 data 初始化，注意此时还没有挂载 html 到页面上。

mounted

挂载完成，也就是模板中的 HTML 渲染到 HTML 页面中，此时一般可以做一些 ajax 操作，mounted 只会执行一次。

- mounted 常用于需要对 DOM 进行操作的任务，例如初始化第三方库、绑定事件监听器、执行动画等。

beforeUpdate

在数据更新之前被调用，发生在虚拟 DOM 重新渲染和打补丁之前，可以在该钩子中进一步地更改状态，不会触发附加地重渲染过程

updated（更新后）

在由于数据更改导致地虚拟 DOM 重新渲染和打补丁只会调用，调用时，组件 DOM 已经更新，所以可以执行依赖于 DOM 的操作，然后在大多数情况下，应该避免在此期间更改状态，因为这可能会导致更新无限循环，该钩子在服务器端渲染期间不被调用

beforeDestroy（销毁前）

在实例销毁之前调用，实例仍然完全可用，

这一步还可以用 this 来获取实例，

一般在这一步做一些重置的操作，比如清除掉组件中的定时器 和 监听的 dom 事件

destroyed（销毁后）

在实例销毁之后调用，调用后，所有的事件监听器会被移出，所有的子实例也会被销毁，该钩子在服务器端渲染期间不被调用

更详细：

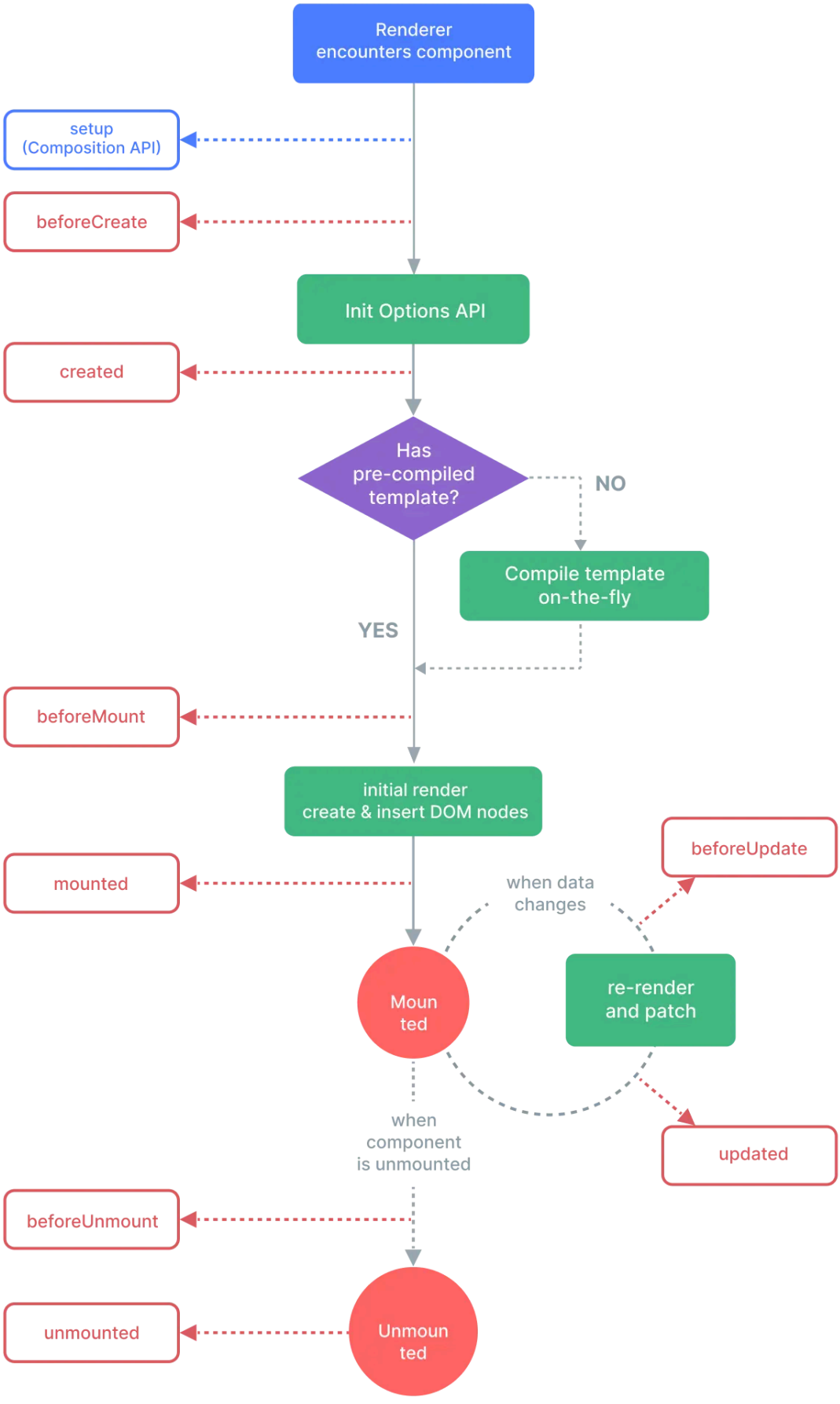
1. 每个 Vue 组件实例被创建后都会经过一系列初始化步骤，比如，它需要数据观测，模板编译，挂载实例到 dom 上，以及数据变化时更新 dom。这个过程中会运行叫做生命周期钩子的函数，以便用户在特定阶段有机会添加他们自己的代码。
2. Vue 生命周期总共可以分为 8 个阶段：创建前后, 载入前后, 更新前后, 销毁前后，以及一些特殊场景的生命周期。 vue3 中新增了三个用于调试和服务端渲染场景。

生命周期v2	生命周期v3	描述
beforeCreate	beforeCreate	组件实例被创建之初
created	created	组件实例已经完全创建
beforeMount	beforeMount	组件挂载之前
mounted	mounted	组件挂载到实例上去之后
beforeUpdate	beforeUpdate	组件数据发生变化，更新之前
updated	updated	数据数据更新之后
beforeDestroy	beforeUnmounted	组件实例销毁之前
destroyed	unmounted	组件实例销毁之后

生命周期v2	生命周期v3	描述
activated	activated	keep-alive缓存的组件激活时
deactivated	deactivated	keep-alive缓存的组件停用时调用
errorCaptured	errorCaptured	捕获一个来自子孙组件的错误时被调用
–	renderTracked	调试钩子，响应式依赖被收集时调用
–	renderTriggered	调试钩子，响应式依赖被触发时调用
–	serverPrefetch	ssr only，组件实例在服务器上被渲染前调用

选项式 API	Hook inside `setup`
`beforeCreate`	Not needed*
`created`	Not needed*
`beforeMount`	`onBeforeMount`
`mounted`	`onMounted`
`beforeUpdate`	`onBeforeUpdate`
`updated`	`onUpdated`
`beforeUnmount`	`onBeforeUnmount`
`unmounted`	`onUnmounted`
`errorCaptured`	`onErrorCaptured`
`renderTracked`	`onRenderTracked`
`renderTriggered`	`onRenderTriggered`
`activated`	`onActivated`
`deactivated`	`onDeactivated`

3、Vue3 生命周期流程图：



@稀土掘金技术社区

4.结合实践：

- beforeCreate：通常用于插件开发中执行一些初始化任务
- created：组件初始化完毕，可以访问各种数据，获取接口数据等
- mounted：dom 已创建，可用于获取访问数据和 dom 元素；访问子组件等。
- beforeUpdate：此时 view 层还未更新，可用于获取更新前各种状态
- updated：完成 view 层的更新，更新后，所有状态已是最新
- beforeunmounted：实例被销毁前调用，可用于一些定时器或订阅的取消
- unmounted：销毁一个实例。可清理它与其它实例的连接，解绑它的全部指令及事件监听器

```
// vue2中声明周期的派发时刻：
...
vm._self = vm
initLifecycle(vm)
initEvents(vm)
initRender(vm)
callHook(vm, 'beforeCreate')
initInjections(vm) // resolve injections before data/props
initState(vm)
initProvide(vm) // resolve provide after data/props
callHook(vm, 'created')
...
...

// vue3中生命周期的派发时刻：
...
// call beforeCreate first before accessing other options since
// the hook may mutate resolved options (#2791)
if (options.beforeCreate) {
  callHook(options.beforeCreate, instance, LifecycleHooks.BEFORE_CREATE)
}
...
...
```

17、computed 和 watch 的区别

计算属性 computed:

1. 支持缓存，只有依赖数据发生改变，才会重新进行计算
2. 不支持异步，当 computed 内有异步操作时无效，无法监听数据的变化
3. computed 属性值会默认走缓存，计算属性是基于它们的响应式依赖进行缓存的，也就是基于 data 中声明过或者父组件传递的 props 中的数据通过计算得到的值
4. 如果一个属性是由其他属性计算而来的，这个属性依赖其他属性，是一个多对一或者一对一，一般用 computed
5. 如果 computed 属性值是函数，那么默认会走 get 方法；函数的返回值就是属性值；
6. 在 computed 中的，属性都有一个 get 和一个 set 方法，当数据变化时，调用 set 方法。

侦听属性 watch：

1. 不支持缓存，数据变，直接会触发相应的操作；
2. watch 支持异步；
3. 监听的函数接收两个参数，第一个参数是最新的值；第二个参数是输入之前的值；
4. 当一个属性发生变化时，需要执行对应的操作；一对多；
5. 监听数据必须是 data 中声明过或者父组件传递过来的 props 中的数据，当数据变化时，触发其他操作，函数有两个参数，
    - immediate：组件加载立即触发回调函数执行，
    - deep: 深度监听，为了发现对象内部值的变化，复杂类型的数据时使用，例如数组中的对象内容的改变，注意监听数组的变动不需要这么做。
    - 注意：deep 无法监听到数组的变动和对象的新增，参考 vue 数组变异,只有以响应式的方式触发才会被监听到。
6. 当需要在数据变化时执行异步或开销较大的操作时，这个方式是最有用的。这是和 computed 最大的区别，请勿滥用.

```
data(){
 return {
   age:5,
   type: 'child'
 }
},
computed: {
 naive(){
   console.log('check is naive')
   if(this.age < 10 || this.type === 'child'){
     return 'yes'
   }
   return  'no'
 }
}
// 更新 this.type 会输出 check is naive 吗?  不会输出！！！
// 看条件判断，重点在条件判断地方
```

我们在组件中使用 computed 计算属性时，当组件初始化的时候系统便会对每个定义的 key 都创建了对应的 watcher, 并有一个特殊的参数 lazy, 然后调用了自己的 getter 方法, 这样就收集了这个计算属性依赖的所有 data。

那么所依赖的 data 会收集这个订阅者同时会针对 computed 中的 key 添加属性描述符创建了独有的 get 方法，当调用计算属性的时候，这个 get 判断 dirty 是否为 true，为真则表示要要重新计算，反之直接返回 value。

当依赖的 data 变化的时候会触发数据的 set 方法调用 update() 通知更新，此时会把 dirty 设置成 true，所以 computed 就会重新计算这个值，从而达到动态计算的目的。

概念上的区别

- **watch**：侦听响应式数据的变化，若数据变化执行副作用函数
- **computed**：由响应式数据派生出的新数据

用法上的区别：

- **watch**：需要明确指定侦听目标，可以获取目标变化前后的值，不需要返回值。
- **computed**：不能指定侦听目标，不能获取目标变化前后的值，需要返回值。

选watch的情况：

- 需要指定确切侦听目标，或需要变化前后的值;
- 响应数据变化时需要做异步操作或耗时操作时。

大部分情况可选 computed。

18、`vue.$set` 和 `vue.$delete`

实例创建后添加属性，并不会触发视图更新

这时候需要使用 vue中 $set 方法,既可以新增属性，又可更新视图

删除对象的 property。如果对象是响应式的，确保删除能触发更新视图。这个方法主要用于避开 Vue 不能检测到 property 被删除的限制，但是你应该很少会使用它。

19、自定义指令

在 Vue 中，自定义指令（Custom Directive）是一种用于扩展 `Vue` 的模板语法的机制。通过自定义指令，你可以在 DOM 元素上添加自定义行为，并在元素插入、更新和移除时进行相应的操作。

自定义指令由 `Vue.directive` 函数定义，它接收两个参数：指令名称和指令选项对象。指令选项对象包含一系列钩子函数，用于定义指令的行为。

以下是一些常见的自定义指令应用场景：

1. **操作 DOM**：自定义指令可以用于直接操作 DOM 元素，例如修改元素的样式、属性、事件绑定等。你可以通过在指令的钩子函数中访问和操作 DOM 元素。
2. **表单验证**：你可以创建自定义指令来实现表单验证逻辑。通过自定义指令，你可以监听输入框的值变化，并根据自定义的验证规则进行验证，以便提供实时的反馈。
3. **权限控制**：自定义指令可以用于权限控制场景，例如根据用户权限来隐藏或禁用某些元素。你可以在自定义指令中根据用户权限进行条件判断，并修改元素的显示或行为。
4. **第三方库集成**：当你需要在 Vue 中使用第三方库或插件时，可以使用自定义指令来进行集成。你可以创建一个自定义指令，在其中初始化和配置第三方库，并在适当的时机调用库的方法。
5. **动画和过渡效果**：自定义指令可以与 Vue 的过渡系统一起使用，实现自定义的动画和过渡效果。你可以在自定义指令中监听过渡钩子函数，并根据需要操作元素的样式或类名来实现过渡效果。

这只是一些常见的应用场景，实际上自定义指令的应用范围非常广泛，可以根据具体需求进行灵活的使用。通过自定义指令，你可以扩展 Vue 的能力，实现更复杂和灵活的交互行为。

更多可参考：juejin.cn/post/721504…

```
// 注册一个全局自定义指令 `v-focus`
Vue.directive('focus', {
  // 当被绑定的元素插入到 DOM 中时……
  inserted: function (el) {
    // 聚焦元素
    el.focus()
  }
})
```

20、keep-alive 的实现

作用：实现组件缓存，保持这些组件的状态，以避免反复渲染导致的性能问题。 需要缓存组件 频繁切换，不需要重复渲染

场景：tabs 标签页 后台导航，vue 性能优化

原理：

- Vue.js 内部将 DOM 节点抽象成了一个个的 VNode 节点，keep-alive 组件的缓存也是基于 VNode 节点的而不是直接存储 DOM 结构。
- 它将满足条件 （pruneCache 与 pruneCache） 的组件在 cache 对象中缓存起来，在需要重新渲染的时候再将 vnode 节点从 cache 对象中取出并渲染。

21、Vue 里 template 到 render 经过哪些步骤

1. 解析模板（Parsing Template） ——> AST
2. 静态分析（Static Analysis）静态节点、动态节点
3. 生成渲染函数（Generate Render Function）
4. 创建虚拟 DOM（Create Virtual DOM）
5. 更新虚拟 DOM（Update Virtual DOM）
6. 实际 DOM 操作（Actual DOM Manipulation）

22、vue 怎么监听数组的?

在将数组处理成响应式数据后，如果使用数组原始方法改变数组时，数组值会发生变化，但是并不会触发数组的setter来通知所有依赖该数组的地方进行更新，为此，vue通过重写数组的某些方法来监听数组变化，重写后的方法中会手动触发通知该数组的所有依赖进行更新。

array.js中重写了数组的push、pop、shift、unshift、splice、sort、reverse七种方法，重写方法在实现时除了将数组方法名对应的原始方法调用一遍并将执行结果返回外，还通过执行ob.dep.notify()将当前数组的变更通知给其订阅者，这样当使用重写后方法改变数组后，数组订阅者会将这边变化更新到页面中。

重写完数组的上述7种方法外，我们还需要将这些重写的方法应用到数组上，因此在Observer构造函数中，可以看到在监听数据时会判断数据类型是否为数组。当为数组时，如果浏览器支持__proto__，则直

接将当前数据的原型__proto__指向重写后的数组方法对象arrayMethods，如果浏览器不支持__proto__，则直接将arrayMethods上重写的方法直接定义到当前数据对象上；当数据类型为非数组时，继续递归执行数据的监听。

23、Vue router 路由实现原理

1. HashHistory

```
window.addEventListener("hashchange", funcRef, false)
```

hash 模式的优缺点：

- **优点**：浏览器兼容性较好，连 IE8 都支持
- **缺点**：路径在井号 # 的后面，比较丑

2. HTML5History

```
window.history.pushState(stateObject, title, URL)
window.history.replaceState(stateObject, title, URL)
popState
```

history 模式的优缺点：

- **优点**：路径比较正规，没有井号 #
- **缺点**：兼容性不如 hash，且需要服务端支持，否则一刷新页面就404了

以最常用的 Nginx 为例，只需要在配置的 `location /` 中增加下面一行即可：

```
`try_files $uri /index.html;`
```

3. transitionTo()函数

```
HashHistory.push()
HashHistory.replace()

HTML5History.pushState()
HTML5History.replaceState()
```

24、vue-router 中如何保护指定路由的安全?

1、全局的钩子函数

- beforeEach(to，from，next) 路由改变前调用。常用验证用户权限。to ： 即将要进入的目标路由对象。from：当前正要离开的路由对象。next：路由控制参数。next()：如果一切正常，则调用这个方法进入下一个钩子。next(false)：取消导航（即路由不发生改变）。next('/login')：当前导航

被中断，然后进行一个新的导航。next(error)：如果一个Error实例，则导航会被终止且该错误会被传递给router.onError()

- afterEach (to，from) 路由改变后的钩子。常用自动让页面返回最顶端。用法相似，少了next参数。

2、路由配置中的导航钩子

- beforeEnter (to，from，next)

3、组件内的钩子函数

- beforeRouteEnter(to,from,next)。该组件的对应路由被comfirm前调用。 此时实例还没被创建，所以不能获取实例（this）
- beforeRouteUpdate(to,from,next)。当前路由改变，当该组件被复用时候调用。该函数内可以访问组件实例(this)。举例来说，对于一个带有动态参数的路径 /foo/:id，在 /foo/1 和 /foo/2 之间跳转的时候，由于会渲染同样的 Foo 组件，因此组件实例会被复用。而这个钩子就会在这个情况下被调用。
- beforeRouteLeave(to,from,next)。当导航离开组件的对应路由时调用。 该函数内可以访问获取组件实例（this）

4、路由监测变化（使用 watch 来对 $route 监听）

- `route` 主要用于获取当前路由信息，
- `router` 则是用于进行路由操作，例如跳转到指定的路由、前进、后退等。
- 监听到路由对象发生变化，从而对路由变化做出响应

1. vue-router 中保护路由安全通常使用导航守卫来做，通过设置路由导航钩子函数的方式添加守卫函数，在里面判断用户的登录状态和权限，从而达到保护指定路由的目的。
2. 具体实现有几个层级:全局前置守卫 beforeEach、路由独享守卫beforeEnter 或组件内守卫 beforeRouteEnter。以全局守卫为例来说，可以使用 router.beforeEach((to,from,next)=> {}) 方式设置守卫，每次路由导航时，都会执行该守卫，从而检查当前用户是否可以继续导航，通过给 next 函数传递多种参数达到不同的目的，比如如果禁止用户继续导航可以传递 next(false)，正常放行可以不传递参数，传递 path 字符串可以重定向到一个新的地址等等。
3. 这些钩子函数之所以能够生效，也和 vue-router 工作方式有关，像beforeEach 只是注册一个 hook，当路由发生变化，router 准备导航之前会批量执行这些 hooks，并且把目标路由 to，当前路由 from，以及后续处理函数 next传递给我们设置的 hook。

25、Vue mixin 作用、原理、覆盖顺序

Mixins：则是在引入组件之后与组件中的对象和方法进行合并，相当于扩展了父组件的对象与方法，可以理解为形成了一个新的组件。

`Mixin` 类通常作为功能模块使用，在需要该功能时“混入”，有利于代码复用又避免了多继承的复杂。

> 当组件存在与 `mixin` 对象相同的选项的时候，进行递归合并的时候组件的选项会覆盖 `mixin` 的选项

但是如果相同选项为生命周期钩子的时候，会合并成一个数组，先执行 mixin 的钩子，再执行组件的钩子

源码位置：/src/core/global-api/mixin.js

```
export function initMixin (Vue: GlobalAPI) {
    Vue.mixin = function (mixin: Object) {
        this.options = mergeOptions(this.options, mixin)
        return this
    }
}
```

主要是调用 merOptions 方法

源码位置：/src/core/util/options.js

```
export function mergeOptions (
  parent: Object,
  child: Object,
  vm?: Component
): Object {

if (child.mixins) { // 判断有没有 mixin 也就是 mixin 里面挂 mixin 的情况 有的话递归进行合并
    for (let i = 0, l = child.mixins.length; i < l; i++) {
    parent = mergeOptions(parent, child.mixins[i], vm)
    }
}

  const options = {}
  let key
  for (key in parent) {
    mergeField(key) // 先遍历 parent 的 key 调对应的 strats[XXX] 方法进行合并
  }
  for (key in child) {
    if (!hasOwn(parent, key)) { // 如果 parent 已经处理过某个 key 就不处理了
      mergeField(key) // 处理 child 中的 key 也就 parent 中没有处理过的 key
    }
  }
  function mergeField (key) {
    const strat = strats[key] || defaultStrat
    // 根据不同类型的 options 调用 strats 中不同的方法进行合并
    options[key] = strat(parent[key], child[key], vm, key)
  }
  return options
}
```

从上面的源码，我们得到以下几点：

- 优先递归处理 `mixins`
- 先遍历合并 `parent` 中的 `key`，调用 `mergeField` 方法进行合并，然后保存在变量 `options`
- 再遍历 `child`，合并补上 `parent` 中没有的 `key`，调用 `mergeField` 方法进行合并，保存在变量 `options`
- 通过 `mergeField` 函数进行了合并

26、vue 的 inject provide 是怎么做的?

1. mergeOptions函数
2. normalizeInject(child, vm);
3. vm.$options.inject = {"parentValue": {"from": "parentValue"}}
4. 实例化子组件时对inject的处理。在_init时会调用initInjections函数
5. 从resolveInject函数可以看到通过while循环，以及source = source.$parent 找到父组件中的_provided属性，拿到其值，**也就拿到父组件提供的provide了**
6. 调用了mergeOptions对父组件中的provide属性进行了处理
7. vm.$options.provide 就是mergedInstanceDataFn函数。通过调用这个函数我们_provided就成为了{"parentValue":"here is parent data"}

27、Vuex 使用及理解

使用：

1. `state` （...mapState 可获取，mutate）
2. `getter` （getter 理解为计算属性）
3. `mutation` （更改 vuex 的 state 中唯一的方式，必须是同步函数，commit）
4. `action` （dispatch、ajax）
    - action 提交的 mutation，不是直接修改状态
    - action 可以包含异步操作，而 mutation 不行
    - action 中的回调函数第一个参数是 context，是一个与 store 实例具有相同属性的方法的对象
    - action 通过 store.dispatch 触发，mutation 通过 store.commit 提交
5. `module` （vuex 允许我们将 store 分割成模块）

Vuex 的辅助函数：

- `state` 辅助函数为 `mapState`，把 state 属性映射到 computed 身上
- `actions` 辅助函数为 `mapActions`，把 actions 里面的方法映射到 methods 中
- `mutations` 辅助函数为 `mapMutations`，把 mutations 里面的方法映射到 methods 中
- `mapGetters`: 把 getters 属性映射到 computed 身上

理解：

1. vuex 是 `vue` 专用的状态管理库。它以全局方式集中管理应用的状态，并且可以保证状态变更的可预测性。

2. vuex 主要解决的问题是多组件之间状态共享的问题，利用各种组件通信方式，我们虽然能够做到状态共享，但是往往需要在多个组件之间保持状态的一致性，这种模式很容易出现问题，也会使程序逻辑变得复杂。vuex 通过把组件的共享状态抽取出来，以全局单例模式管理，这样任何组件都能用 一致的方式获取和修改状态，响应式的数据也能够保证简洁的单向数据流动，我们的代码将变得更结构化且易维护。
3. vuex 并非必须的，它帮我们管理共享状态，但却带来更多的概念和框架。如果我们不打算开发大型 单页应用或者我们的应用并没有大量全局的状态需要维护，完全没有使用 vuex 的必要。一个简单的 store 模式就足够了。反之，Vuex 将会成为自然而然的选择。引用 Redux 的作者 Dan Abramov 的话说就是: `Flux 架构就像眼镜：您自会知道什么时候需要它。`
4. 我在使用 vuex 过程中有如下理解:
    - 首先是对核心概念的理解和运用，将全局状态放入 state 对象 中，它本身一棵状态树，组件中使用 store 实例的 state 访问这些状态;
    - 然后有配套的 mutation 方法 修改这些状态，并且只能用 mutation 修改状态，在组件中调用 commit 方法提交 mutation;
    - 如果应 用中有异步操作或者复杂逻辑组合，我们需要编写 action，执行结束如果有状态修改仍然需要提交 mutation，组件中调用这些 action 使用 dispatch 方法派发。
    - 最后是模块化，通过 modules 选项组织 拆分出去的各个子模块，在访问状态时注意添加子模块的名称，如果子模块有设置 namespace，那 么在提交 mutation 和派发 action 时还需要额外的命名空间前缀。
5. vuex 在实现单项数据流时需要做到数据的响应式，通过源码的学习发现是借用了 `vue` 的数据响应化 特性实现的，它会利用 Vue 将 state 作为 data 对其进行响应化处理，从而使得这些状态发生变化时，能够导致组件重新渲染。

28、vuex 为什么不是响应式的

原来获取 vuex 中的值一定要用计算属性获取

29、Vue 中如何扩展一个组件

1. 常见的组件扩展方法有：`mixins，slots，extends` 等
2. 混入 mixins 是分发 Vue 组件中可复用功能的非常灵活的方式。混入对象可以包含任意组件选项。当组件使用混入对象时，所有混入对象的选项将被混入该组件本身的选项。

```
// 复用代码：它是一个配置对象，选项和组件里面一样
const mymixin = {
  methods: {
    dosomething(){}
  }
}
// 全局混入：将混入对象传入
Vue.mixin(mymixin)
// 局部混入：做数组项设置到mixins选项，仅作用于当前组件
```

```
const Comp = {
  mixins: [mymixin]
}
```

3. 插槽主要用于 vue 组件中的内容分发，也可以用于组件扩展。
    - 如果要精确分发到不同位置可以使用具名插槽，如果要使用子组件中的数据可以使用作用域插槽。
4. 组件选项中还有一个不太常用的选项 extends，也可以起到扩展组件的目的。

```
// 扩展对象
const myextends = {
  methods: {
    dosomething(){}
  }
}
// 组件扩展：做数组项设置到extends选项，仅作用于当前组件
// 跟混入的不同是它只能扩展单个对象
// 另外如果和混入发生冲突，该选项优先级较高，优先起作用
const Comp = {
  extends: myextends
}
```

5. 混入的数据和方法不能明确判断来源且可能和当前组件内变量产生命名冲突，vue3 中引入的 composition api，可以很好解决这些问题，利用独立出来的响应式模块可以很方便的编写独立逻辑并提供响应式的数据，然后在 setup 选项中组合使用，增强代码的可读性和维护性。例如：

```
// 复用逻辑1
function useXX() {}
// 复用逻辑2
function useYY() {}
// 逻辑组合
const Comp = {
  setup() {
    const {xx} = useXX()
    const {yy} = useYY()
    return {xx, yy}
  }
}
```

30、子组件可以直接改变父组件的数据吗?

1. 所有的 prop 都使得其父子之间形成了一个单向下行绑定：父级 prop 的更新会向下流动到子组件中，但是反过来则不行。这样会防止从子组件意外变更父级组件的状态，从而导致你的应用的数据流向难以理解。另外，每次父级组件发生变更时，子组件中所有的 prop 都将会刷新为最新的值。这意味着你不应该在一个子组件内部改变 prop。如果你这样做了，Vue 会在浏览器控制台中发出警告。

```
const props = defineProps(['foo'])
// 下面行为会被警告，props是只读的!
props.foo = 'bar'
```

2. 实际开发过程中有两个场景会想要修改一个属性：

- 这个 prop 用来传递一个初始值；这个子组件接下来希望将其作为一个本地的 prop 数据来使用。在这 种情况下，最好定义一个本地的 data，并将这个 prop 用作其初始值：

```
const props = defineProps(['initialCounter'])
const counter = ref(props.initialCounter)
```

- 这个 prop 以一种原始的值传入且需要进行转换。在这种情况下，最好使用这个 prop 的值来定义一个计 算属性：

```
const props = defineProps(['size'])
// prop变化，计算属性自动更新
const normalizedSize = computed(() => props.size.trim().toLowerCase())
```

3. 实践中如果确实想要改变父组件属性应该emit一个事件让父组件去做这个变更。注意虽然我们不能直接修改 一个传入的对象或者数组类型的prop，但是我们还是能够直接改内嵌的对象或属性。

31、new Vue() 过程发生了什么?

- new Vue的时候调用会调用_init方法
  - 定义 $set、 $get 、$delete、$watch 等方法
  - 定义 $on、$off、$emit、$off 等事件
  - 定义 _update、$forceUpdate、$destroy生命周期
- 调用$mount进行页面的挂载
- 挂载的时候主要是通过mountComponent方法
- 定义updateComponent更新函数
- 执行render生成虚拟DOM
- _update将虚拟DOM生成真实DOM结构，并且渲染到页面中

32、v-show 和 v-if 区别

1. 编译时刻 vs 运行时刻：
   - v-if 是一个“惰性”指令，在编译时刻，Vue.js 会根据条件决定是否编译或挂载元素到 DOM 中。如果条件为 false，元素根本不会被编译和渲染到 DOM 中。
   - v-show 是一个“非惰性”指令，在编译时刻，元素总是会被编译和渲染到 DOM 中。但是，根据条件的值，v-show 会通过 CSS 控制元素的显示和隐藏，不会从 DOM 中移除元素。
2. 显示隐藏方式：

    - `v-if` 在条件为 `true` 时会渲染元素到 DOM，而在条件为 `false` 时会从 DOM 中移除元素。`v-if` 也可以触发组件创建和销毁的生命钩子。
    - `v-show` 在条件为 `true` 时会通过 CSS 设置元素的 `display` 属性为可见（通常是 `display: block`），在条件为 `false` 时设置为隐藏（`display: none`）。元素始终存在于 DOM 中，只是通过 CSS 控制其显示状态。
3. 切换开销：
    - `v-if` 在条件切换时，如果条件从 `true` 切换为 `false`，会销毁并重新创建元素，这涉及到 DOM 的删除和重新插入，可能会有一定的性能开销。
    - `v-show` 在条件切换时，只是简单地通过 CSS 控制元素的显示和隐藏，不会销毁和重新创建元素，因此切换的开销较小。
4. 初始渲染开销：
    - `v-if` 在初始渲染时，如果条件为 `false`，元素不会被渲染到 DOM 中，因此在初始渲染时可能会有一定的性能优势。
    - `v-show` 在初始渲染时，元素总是会被渲染到 DOM 中，因此在初始渲染时可能会有一些额外的开销。

- 当需要频繁切换元素的显示状态时，且元素可能处于不同的状态，推荐使用 `v-show`。
- 而当条件不会频繁改变，且希望在条件为 `false` 时不渲染元素到 DOM 中，推荐使用 `v-if`。

33、vue3 新特性

1. 更快
    - 虚拟 `DOM` 重写（编译时提示减少运行时开销，使用更有效的代码创建虚拟节点。组件快速路径 + 单个调用+子节点类型检查。跳过不必要的条件分支。js 引擎更容易优化）
    - 优化 `slots` 的生成（vue3 中可以单独重新渲染父级和子级。确保实例正确的跟踪依赖关系。避免不必要的父子组件重新渲染）
    - 静态树提升（内存换时间，Vue3 的编译器将能够检测到什么是静态的，然后将其提升，从而降低了渲染成本。跳过修补整棵树，从而降低渲染成本。即使多次出现也能正常工作 ）
    - 静态属性提升（Vue3 打补丁时将跳过这些属性不会改变的节点）
    - 基于 `Proxy` 的响应式系统 （组件实例初始化的速度提高 100% 。使用 Proxy 节省以前一半的内存开销，加快速度，但是存在低浏览器版本的不兼容。为了继续支持 IE11，Vue3 将发布一个支持旧观察者机制和新 Proxy 版本的构建）
    - vue-next-template-explorer.netlify.app/
2. 更小
    - 通过摇树优化核心库体积
3. 更容易维护
    - TS+模块化 （它不仅会使用 TypeScript，而且许多包被解耦，更加模块化。）
4. 更加友好
    - 跨平台：编译器核心和运行时核心与平台无关
5. 更容易使用
    - 改进 ts 支持，编译器提供更好的类型检查和错误及警告
    - 更好的调试支持
    - 独立的响应式模块

- Composition API
- 引入 Teleport 组件：Vue 3 引入了 Teleport 组件，可以在 DOM 树中的不同位置渲染内容，用于创建模态框、工具提示和其他覆盖层效果。
- 片段（Fragments）：Vue 3 引入了一个名为片段（Fragment）的内置组件，允许将多个元素进行分组，而无需添加额外的包装元素。

34、在 Vue3 优雅的使用 v-model

在Vue2.0中如何实现双向数据绑定：

一种是v-model，另一种是.sync。

因为一个组件只能用于一个v-model，但是有的组件需要有多个可以双向响应的数据，所以就出现了.sync。

在Vue3.0中为了实现统一，实现了让一个组件可以拥有多个v-model，同时删除掉了.sync。

在vue3.0中，v-model后面需要跟一个modelValue，即要双向绑定的属性名，Vue3.0就是通过给不同的v-model指定不同的modelValue来实现多个v-model。

参考地址: v3.vuejs.org/guide/migra…

35、Vue3 中怎么设置全局变量?

1. `config.globalProperties`
   - `vue2.x` 挂载全局是使用 `Vue.prototype.$xxxx=xxx` 的形式来挂载，然后通过 `this.$xxx`来获取挂载到全局的变量或者方法。
   - 在 `Vue 3` 中，就等同于 `config.globalProperties`。这些 `property` 将被复制到应用中作为实例化组件的一部分。

```
// 之前 (Vue 2.x)
Vue.prototype.$http = () => {}
// 之后 (Vue 3.x)
const app = createApp({})
app.config.globalProperties.$http = () => {}
```

2. `Provide / Inject`
   - vue3 新的 `provide/inject` 功能可以穿透多层组件，实现数据从父组件传递到子组件。
   - 可以将全局变量放在根组件的 `provide` 中，这样所有的组件都能使用到这个变量。
   - 如果需要变量是响应式的，就需要在 `provide` 的时候使用 `ref` 或者 `reactive` 包装变量。

作者：雨夜寻晴天 链接：https://juejin.cn/post/7054488305659805727 来源：稀土掘金 著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。